

-- □ View all available databases

```
SHOW DATABASES;
```

-- □ Switch to the project-specific database

```
USE retail_db;
```

-- □ Show all tables in the current database

```
SHOW TABLES;
```

-- □ Preview sample records from the main retail data table

```
SELECT * FROM retail_data;
```

-- □ Total Inventory per Store

-- Helps identify which stores carry the most inventory

```
SELECT
    Store_ID,
    ROUND(SUM(Inventory_Level), 2) AS Total_Inventory
FROM
    retail_data
GROUP BY
    Store_ID
ORDER BY
    Total_Inventory DESC;
```

-- □ Total Inventory per Product

-- Understand which products have the highest stock overall

```
SELECT
    Product_ID,
    ROUND(SUM(Inventory_Level), 2) AS Total_Inventory
FROM
    retail_data
GROUP BY
    Product_ID
ORDER BY
    Total_Inventory DESC;
```

-- □ Inventory per Category and Region

-- Useful for regional managers to assess stock distribution across categories

```

SELECT
    Region,
    Category,
    ROUND(SUM(Inventory_Level), 2) AS Total_Inventory
FROM
    retail_data
GROUP BY
    Region, Category
ORDER BY
    Region, Total_Inventory DESC;

```

-- □ Inventory per Product in Each Store

-- Determine which products are heavily stocked at which store

```

SELECT
    Store_ID,
    Product_ID,
    ROUND(SUM(Inventory_Level), 2) AS Total_Inventory
FROM
    retail_data
GROUP BY
    Store_ID, Product_ID
ORDER BY
    Store_ID, Total_Inventory DESC;

```

-- □ Inventory per Category in Each Store

-- Category-wise inventory snapshot at each store

```

SELECT
    Store_ID,
    Category,
    ROUND(SUM(Inventory_Level), 2) AS Total_Inventory
FROM
    retail_data
GROUP BY
    Store_ID, Category
ORDER BY
    Store_ID, Total_Inventory DESC;

```

-- □ Reorder Point Estimation Using Moving Average and Category-based Lead Time

-- Ensures you can detect which products are nearing low inventory based on future demand

-- Step 1: 5-day moving average of past sales for demand forecast

```
WITH demand_window AS (  
  SELECT  
    STR_TO_DATE(Date, '%Y-%m-%d') AS date_converted,  
    Store_ID,  
    Product_ID,  
    Category,  
    Inventory_Level,  
    Units_Sold,  
    ROUND(  
      AVG(Units_Sold) OVER (  
        PARTITION BY Store_ID, Product_ID  
        ORDER BY STR_TO_DATE(Date, '%Y-%m-%d')  
        ROWS BETWEEN 5 PRECEDING AND 1 PRECEDING  
      ), 2  
    ) AS avg_daily_demand  
  FROM retail_data  
)
```

-- Step 2: Assign category-based lead time

```
add_lead_time AS (  
  SELECT *,  
    CASE  
      WHEN Category IN ('Toys', 'Clothing', 'Groceries') THEN 3  
      WHEN Category IN ('Furniture', 'Electronics') THEN 5  
      ELSE 4  
    END AS lead_time  
  FROM demand_window  
)
```

-- Step 3: Calculate reorder point = avg daily demand × lead time

```
reorder_calc AS (  
  SELECT  
    date_converted AS Date,  
    Store_ID,  
    Product_ID,  
    Category,  
    Inventory_Level,  
    avg_daily_demand,  
    lead_time,  
    ROUND(avg_daily_demand * lead_time, 2) AS reorder_point  
  FROM add_lead_time  
  WHERE avg_daily_demand IS NOT NULL  
)
```

-- Step 4: Flag low inventory risk

```
SELECT  
  Date,
```

```

Store_ID,
Product_ID,
Category,
Inventory_Level,
avg_daily_demand,
lead_time,
reorder_point,
CASE
    WHEN Inventory_Level < reorder_point THEN 'Yes'
    ELSE 'No'
END AS Low_Inventory_Risk
FROM reorder_calc
ORDER BY Date
LIMIT 1000;

```

-- □ Weekly Inventory Turnover for Each Product per Store

-- A key metric to understand how frequently inventory is sold and replaced

```

SELECT
    YEARWEEK(STR_TO_DATE(Date, '%Y-%m-%d'), 1) AS Week,
    Product_ID,
    Store_ID,
    ROUND(SUM(Units_Sold) / NULLIF(AVG(Inventory_Level), 0), 2) AS weekly_turnover
FROM retail_data
GROUP BY Week, Product_ID, Store_ID
ORDER BY weekly_turnover DESC;

```

-- □ Quartile Classification of Weekly Inventory Turnover

-- Helps categorize products into performance quartiles based on turnover rate

```

WITH weekly_turnover_data AS (
    SELECT
        YEARWEEK(STR_TO_DATE(Date, '%Y-%m-%d'), 1) AS Week,
        Product_ID,
        Store_ID,
        ROUND(SUM(Units_Sold) / NULLIF(AVG(Inventory_Level), 0), 2) AS turnover
    FROM retail_data
    GROUP BY Week, Product_ID, Store_ID
),

```

-- Step 2: Rank products into quartiles

```

ranked_turnover AS (
    SELECT
        *,
        NTILE(4) OVER (ORDER BY turnover) AS quartile

```

```
FROM weekly_turnover_data
)
```

-- Step 3: Summary stats for each quartile

```
SELECT
    quartile,
    ROUND(AVG(turnover), 2) AS avg_turnover,
    MIN(turnover) AS min_turnover,
    MAX(turnover) AS max_turnover
FROM ranked_turnover
GROUP BY quartile
ORDER BY quartile;
```

-- □ Categorize Products Based on Turnover Speed

-- Step 1: Weekly turnover per product-store-category

```
WITH weekly_turnover_data AS (
    SELECT
        CONCAT(YEAR(STR_TO_DATE(Date, '%Y-%m-%d')), '-W',
        LPAD(WEEK(STR_TO_DATE(Date, '%Y-%m-%d'), 1), 2, '0')) AS Week,
        Product_ID,
        Store_ID,
        Category,
        ROUND(SUM(Units_Sold) / NULLIF(AVG(Inventory_Level), 0), 2) AS turnover
    FROM retail_data
    GROUP BY Week, Product_ID, Store_ID, Category
),
```

-- Step 2: Label as slow, medium, fast-moving

```
labeled_turnover AS (
    SELECT
        Week,
        Product_ID,
        Store_ID,
        Category,
        turnover,
        CASE
            WHEN turnover <= 4.30 THEN 'Slow-Moving'
            WHEN turnover <= 4.95 THEN 'Medium-Moving'
            ELSE 'Fast-Moving'
        END AS Turnover_Category
    FROM weekly_turnover_data
)
```

-- Final output

```
SELECT *
FROM labeled_turnover
ORDER BY turnover DESC
LIMIT 1000;
```

-- □ Stockout Rate (How often demand exceeds inventory)

-- Overall Stockout Rate

```
SELECT
    ROUND(100.0 * SUM(CASE WHEN Units_Sold > Inventory_Level THEN 1 ELSE 0 END)
    / COUNT(*), 2) AS stockout_rate_percent
FROM retail_data;
```

-- Top 20 Products with Highest Stockout Rates

```
SELECT
    Product_ID,
    ROUND(
        100.0 * SUM(CASE WHEN Units_Sold > Inventory_Level THEN 1 ELSE 0 END) /
COUNT(*),
        2
    ) AS stockout_rate_percent
FROM retail_data
GROUP BY Product_ID
ORDER BY stockout_rate_percent DESC
LIMIT 20;
```

-- Stockout Rate by Store

```
SELECT
    Store_ID,
    ROUND(
        100.0 * SUM(CASE WHEN Units_Sold > Inventory_Level THEN 1 ELSE 0 END) /
COUNT(*),
        2
    ) AS stockout_rate_percent
FROM retail_data
GROUP BY Store_ID
ORDER BY stockout_rate_percent DESC;
```

-- Stockout Rate by Category

```
SELECT
    Category,
    ROUND(
        100.0 * SUM(CASE WHEN Units_Sold > Inventory_Level THEN 1 ELSE 0 END) /
COUNT(*),
        2
    ) AS stockout_rate_percent
```

```
FROM retail_data
GROUP BY Category
ORDER BY stockout_rate_percent DESC;
```

-- Stockout Rate by Region

```
SELECT
    Region,
    ROUND(
        100.0 * SUM(CASE WHEN Units_Sold > Inventory_Level THEN 1 ELSE 0 END) /
        COUNT(*),
        2
    ) AS stockout_rate_percent
FROM retail_data
GROUP BY Region
ORDER BY stockout_rate_percent DESC;
```

-- Top 20 Store-Product combinations with highest stockout rate

```
SELECT
    Product_ID,
    Store_ID,
    ROUND(
        100.0 * SUM(CASE WHEN Units_Sold > Inventory_Level THEN 1 ELSE 0 END) /
        COUNT(*),
        2
    ) AS stockout_rate_percent
FROM retail_data
GROUP BY Product_ID, Store_ID
HAVING stockout_rate_percent > 0
ORDER BY stockout_rate_percent DESC
LIMIT 20;
```

-- □ Average Inventory Levels

-- By Product

```
SELECT
    Product_ID,
    ROUND(AVG(Inventory_Level), 2) AS avg_inventory_level
FROM retail_data
GROUP BY Product_ID
ORDER BY avg_inventory_level DESC
LIMIT 20;
```

-- By Store

```
SELECT
    Store_ID,
```

```
    ROUND(AVG(Inventory_Level), 2) AS avg_inventory_level
FROM retail_data
GROUP BY Store_ID
ORDER BY avg_inventory_level DESC;
```

-- By Category

```
SELECT
    Category,
    ROUND(AVG(Inventory_Level), 2) AS avg_inventory_level
FROM retail_data
GROUP BY Category
ORDER BY avg_inventory_level DESC;
```

-- By Region

```
SELECT
    Region,
    ROUND(AVG(Inventory_Level), 2) AS avg_inventory_level
FROM retail_data
GROUP BY Region
ORDER BY avg_inventory_level DESC;
```

-- By Product and Store Combo

```
SELECT
    Product_ID,
    Store_ID,
    ROUND(AVG(Inventory_Level), 2) AS avg_inventory_level
FROM
    retail_data
GROUP BY Product_ID , Store_ID
ORDER BY avg_inventory_level DESC
LIMIT 20;
```